Tema 4. Eventos y CQRS Asíncrono

Desacoplando y ordenando el tiempo

Recap: DDD + Hexagonal + CQRS

Hemos visto:

- DDD → Modelamos el dominio.
- Hexagonal → Aislamos el dominio.
- CQRS → Separamos lectura y escritura.
- Pero, todo sigue siendo síncrono.

¿Qué significa acoplamiento temporal?

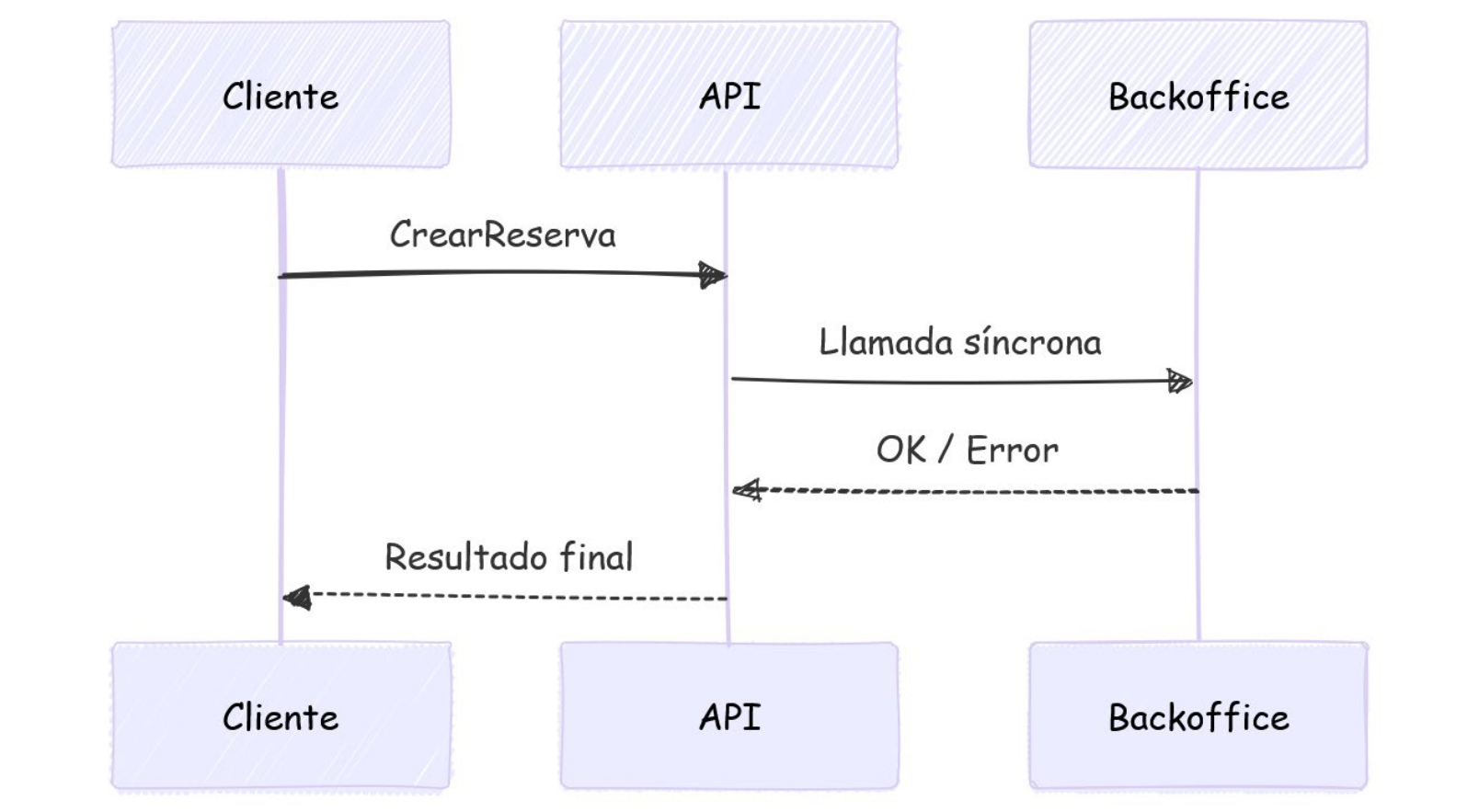
Un sistema está temporalmente acoplado cuando:

- A necesita que B responda ahora.
  - Si B falla, A falla.
  - Si B no responde, A falla.
  - Si B tarda, A bloquea.

Ejemplo real:

API → Backoffice → Respuesta

Diagrama: sistema síncrono clásico



Cambio de mentalidad

Síncrono:

- Ejecutar y esperar.

Asíncrono:

- Emitir intención.
- Procesar después.

Aceptar la intención ≠ Operación completada

Acknowledge vs Resultado Final

- En sistemas síncronos. El cliente recibe:
  ✓ Resultado real
  ✓ Estado final confirmado
- En sistemas asíncronos. El cliente recibe:
  ✓ Confirmación de recepción (ACK)
  ✗ No el resultado final
- Cambio de mentalidad:
  ✗ “Solicitud/Reserva realizada”
  ✓ “Solicitud/Reserva registrada”

Acknowledge (ACK)

ACK significa:

- “He recibido tu mensaje.”

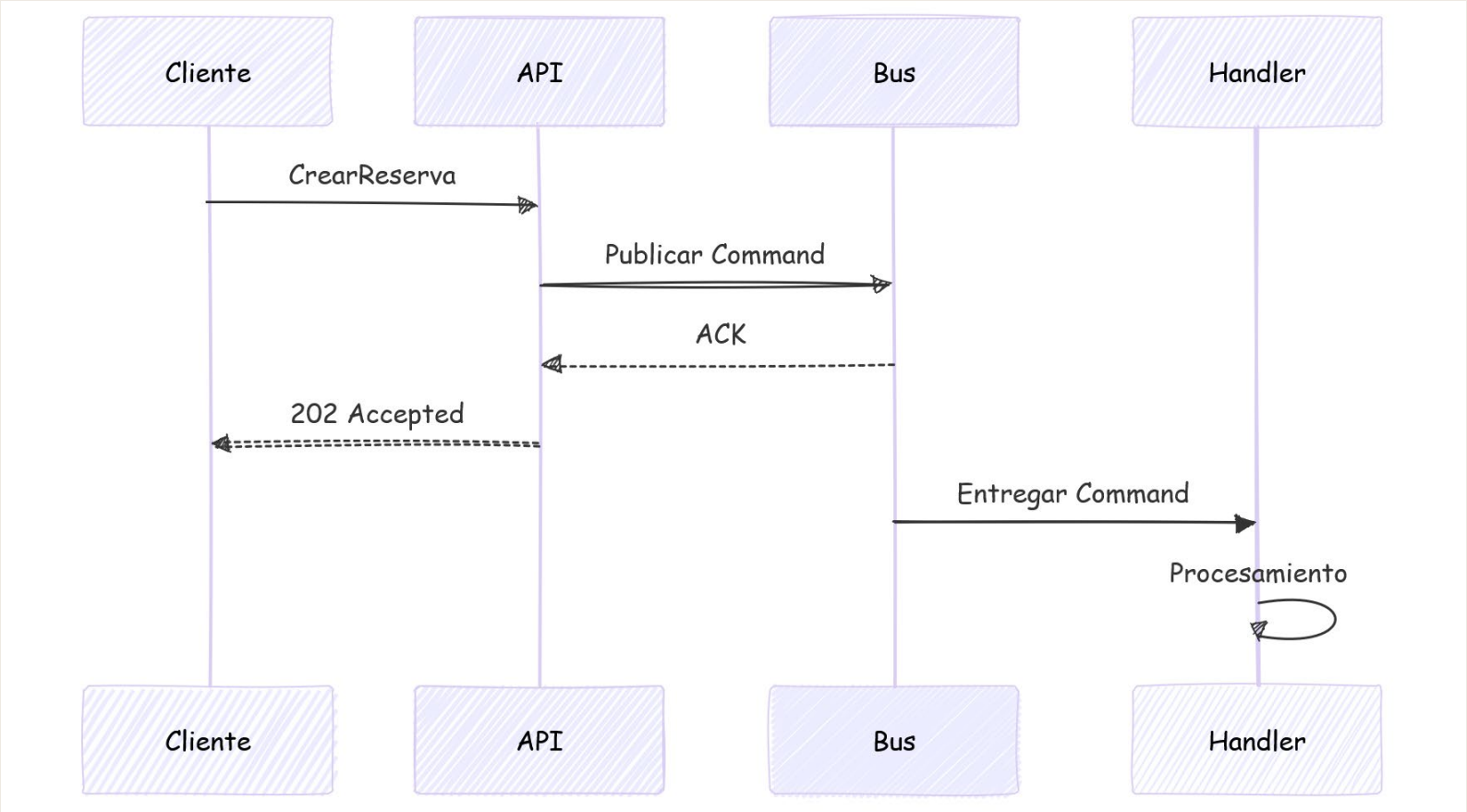
NO significa:

- Que esté procesado.
- Que esté confirmado.
- Que haya tenido éxito.

Ejemplo: Cuando envías un email:

- Tu cliente SMTP responde: “Email accepted”.
- Eso no significa que el destinatario lo haya recibido.
- Solo significa que el servidor lo ha aceptado.

Diagrama: síncrono vs asíncrono



Resultado final diferido

El resultado puede llegar:

- vía evento de resultado
- vía consulta posterior
- vía notificación

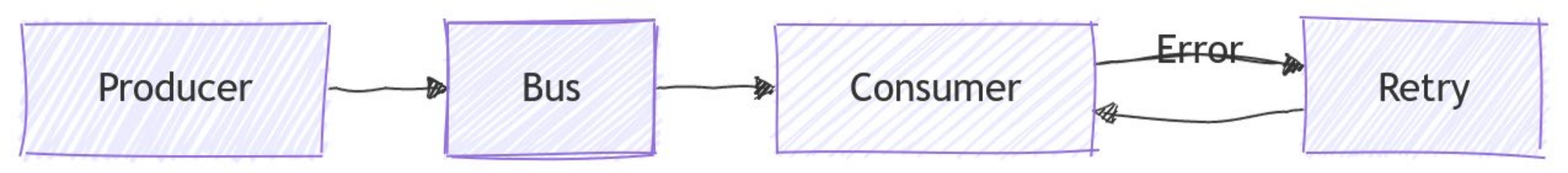
Nosotros lo haremos vía evento.

¿Qué es la semántica de entrega?

Política de entregar un mensaje. Define cuántas veces puede entregarse un mensaje.

- At most once “como mucho una vez”:
  - ✓ 0 o 1 entrega.
  - ✓ Nunca duplicado.
  - ✗ Puede perderse (Pérdida silenciosa).
- At least once “al menos una vez” (es la más común en sistemas reales):
  - ✓ 1 o más entregas (si el bróker no recibe confirmación).
  - ✓ No se pierde.
  - ✗ Puede duplicarse.
- Exactly once “exactamente una vez”:
  - ✓ Ideal teórico.
  - ✓ Se entrega exactamente 1 vez.
  - ✗ Extremadamente complejo en sistemas distribuidos.
  - ⚠ Normalmente es una ilusión implementada sobre at-least-once + idempotencia.

Diagrama de reintento



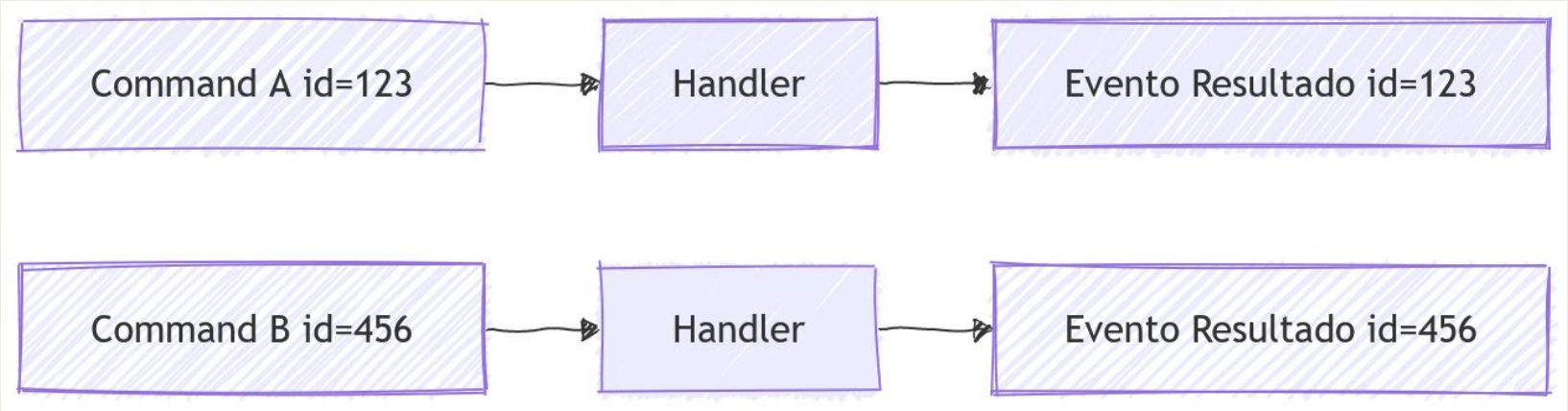
Nosotros usamos **at-least-once**, lo que implica:

- Idempotencia → Porque pueden llegar duplicados.
- Atomicidad local → Sí, todo como una unidad coherente.
- Atomicidad distribuida → No, no podemos tener FO+Broker+BO en una única Transacción.

"En sistemas con semántica at-least-once, la idempotencia no es opcional."

Problema

- Si el resultado llega más tarde…
  - ¿Cómo sabemos a qué comando pertenece?
  - ¿Cómo los relacionamos?



- Por ejemplo:
  - Si llegan desordenados, ¿cómo los relacionamos?

Solución: Correlation ID

Cada command lleva:

- CommandId (EventId)
- CorrelationId
- Y los eventos generados incluyen ese mismo CorrelationId.

- El CorrelationId permite:
  - Trazabilidad
  - Debugging
  - Relacionar intención y resultado
  - Seguir flujos distribuidos

Solución: Correlation ID (EJEMPLO)

Cuando haces un pedido online:

- Te dan un número de pedido.
- Más tarde recibes un email de confirmación.
- Ese email incluye el mismo número.
- Si quieres hacer cualquier actualización necesitas el número de pedido.

Ese número es el correlation Id

¿Qué es un evento? Enfoque purista vs enfoque UMU

Un evento describe algo que ha ocurrido.

Purista	Enfoque UMU
Command expresa intención	Evento puede expresar intención
Evento describe hecho pasado	También usamos evento de intención

⚠️ Nosotros utilizamos eventos como mecanismo de desacoplamiento.

¿Qué es un evento? Enfoque purista vs enfoque UMU

Nosotros:

1. Emitimos eventos de intención para desacoplar
2. El consumidor procesa
3. Luego emite evento de resultado

Es un patrón híbrido entre:

- Command asíncrono
- Event-driven

Alcance de eventos

Evento Local

- Dentro del mismo proceso.

Evento Interno Distribuido

- Entre servicios internos.

Evento de Integración

- Público y versionado.

TIPOS: Eventos del dominio

Representa un hecho relevante del dominio. Características:

- Lenguaje ubicuo
- Semántica de negocio
- Independiente de tecnología

Ejemplo:

- ReservaCreada
- ParkingCancelado

TIPOS: Eventos de integración

Publicado hacia otros sistemas

- Sale fuera de tu bounded context
- Contrato estable
- Versionado
- Puede ser diferente de los eventos del dominio

Ejemplo:

- ReservaCreadaV1
- ReservaCreadaPublica

TIPOS: Eventos de intención

Expresa intención de ejecutar algo. Se usa cuando:

- Queremos desacoplar sincronía
- Queremos procesamiento diferido

Ejemplo:

- SolicitarReserva
- CalcularTarifa

TIPOS: Eventos Technical/Application

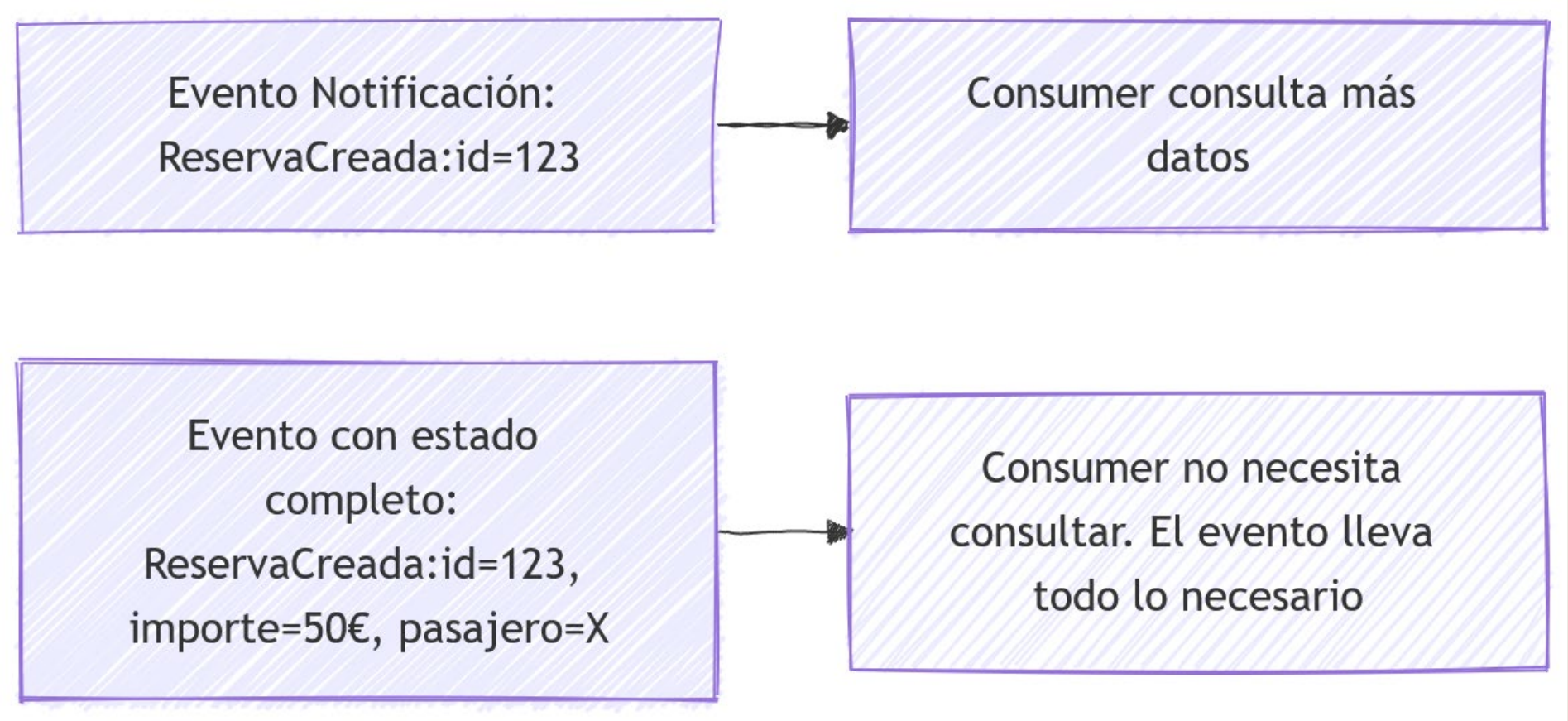
No pertenece al dominio. Sirve para:

- Observabilidad
- Monitorización
- Soporte operativo

Ejemplos:

- MensajeReintentado
- MensajeEnDeadLetter
- EmailEnviado

Notificación VS Estado completo



¿Qué es consistencia eventual?

El sistema puede estar temporalmente inconsistente.

- Pero al final… converge a un estado correcto.
- Durante ese intervalo -> Lecturas pueden no reflejar el cambio inmediato.

Ejemplo. Reservas vuelo:

1. Usuario crea reserva.
2. El evento aún no ha actualizado la proyección.
3. La consulta aún no muestra la reserva.
4. Milisegundos después sí aparece.

Consistencia eventual: Problema

¿Qué pasa si?

- ✓ Se guarda en BBDD
- ✗ Falla el envío al broker
- ⚠ Te quedas con datos persistidos, pero **sin evento publicado**.

O al revés:

- ✗ Falla la BBDD
- ✓ Se publica el evento
- ⚠ Otros servicios creen que existe una reserva que realmente no existe.

Esto rompe la consistencia del sistema distribuido -> Solución: Patrón **OutBOX**

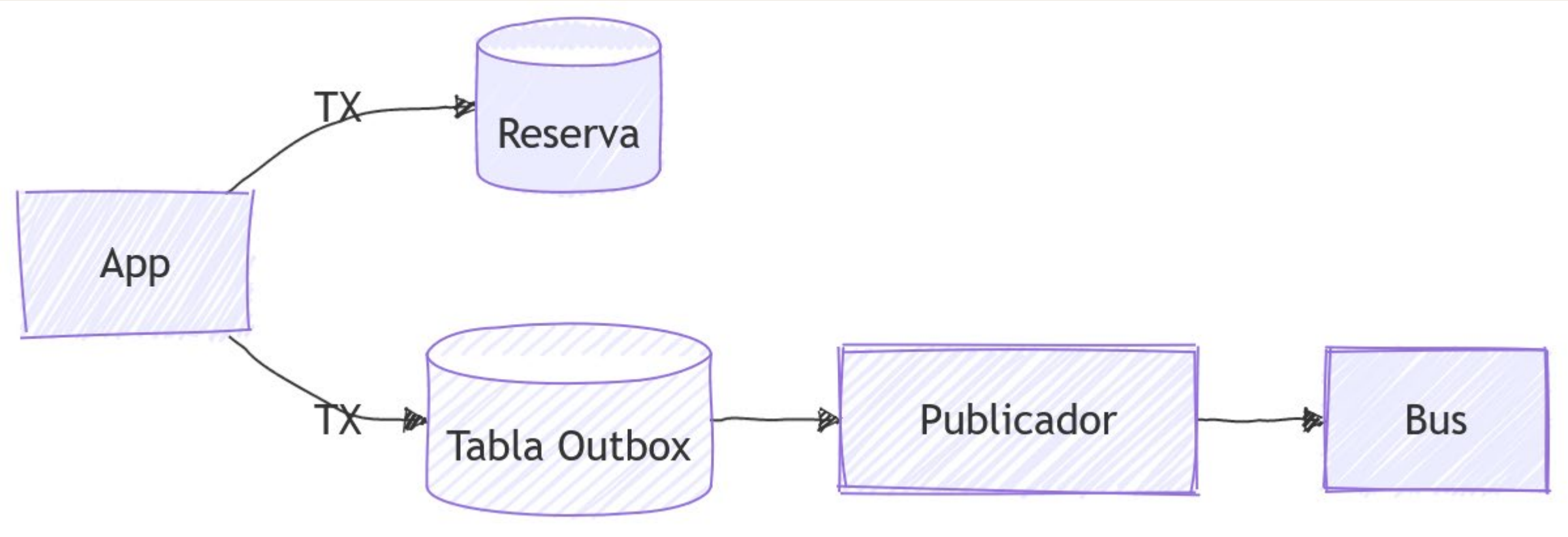
Patrón OutBox

En vez de publicar el evento directamente al broker, lo que hacemos es:

1. Guardar la entidad (ej.: reserva)
2. Guardar el evento en una tabla “outbox”
3. Todo dentro de la **misma transacción**

Después, un proceso independiente se encarga de leer esa tabla y publicar los eventos.

Patrón OutBox: Diagrama



Retries y Dead Letter

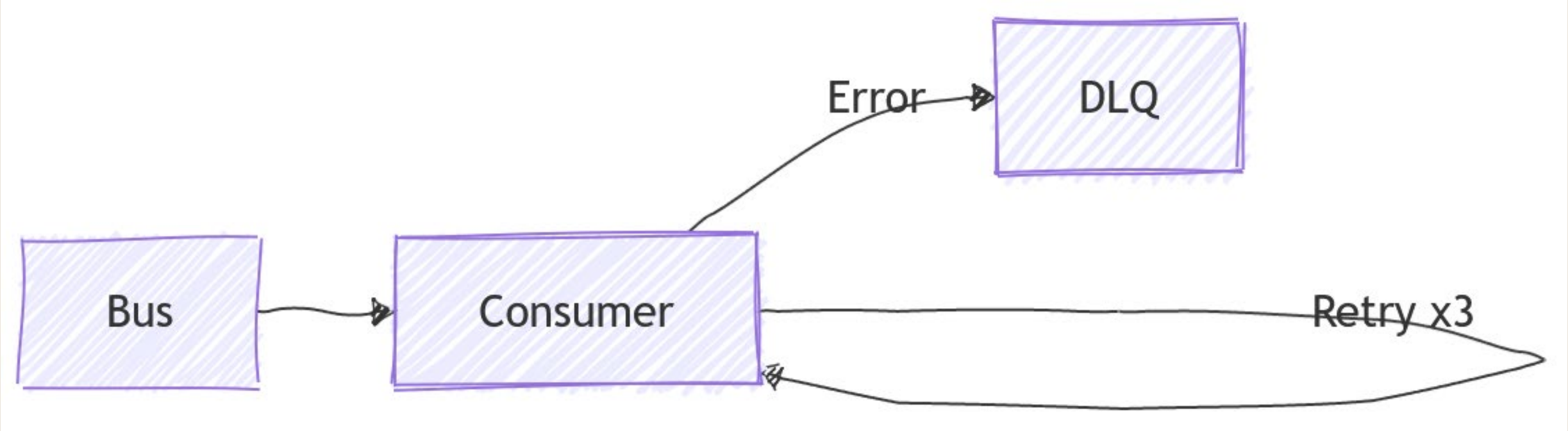
Retry: Reintento automático ante fallo. Si el consumidor falla:

- Mensaje → Error → Reintento
- Puede haber:
  - Backoff exponencial
  - Límite de intentos

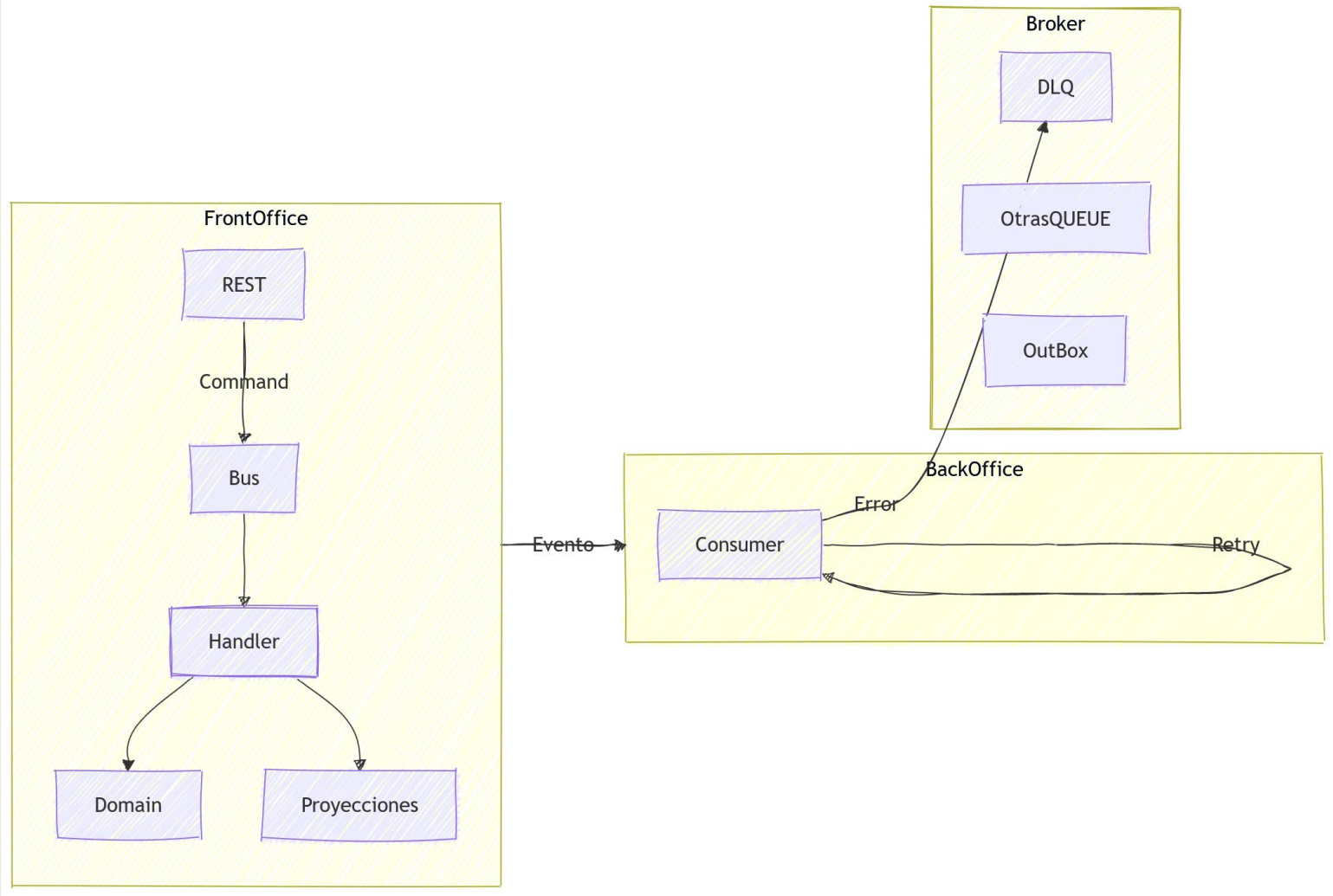
Dead Letter Queue (DLQ): Si tras N reintentos sigue fallando:

- Mensaje → DLQ
- No se pierde.
- Se analiza manualmente.

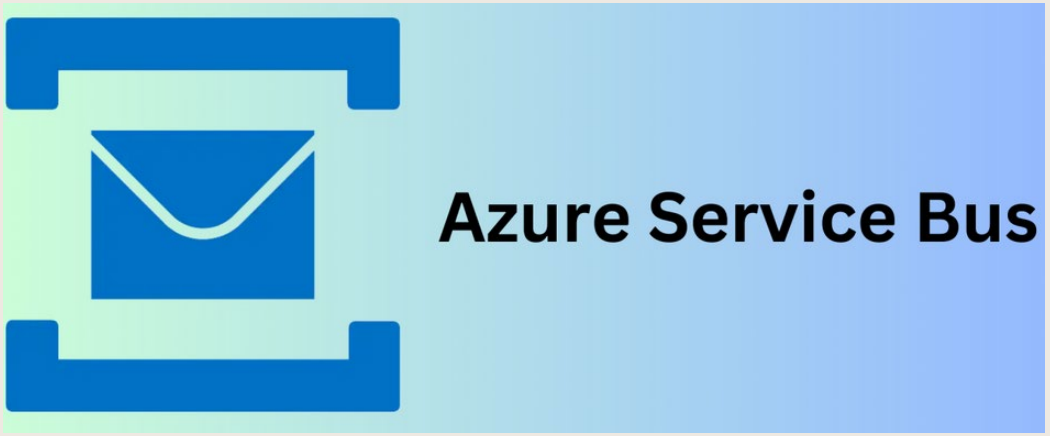
Retries y Dead Letter: Diagrama



Flujo completo



Implementaciones reales de Message Brokers



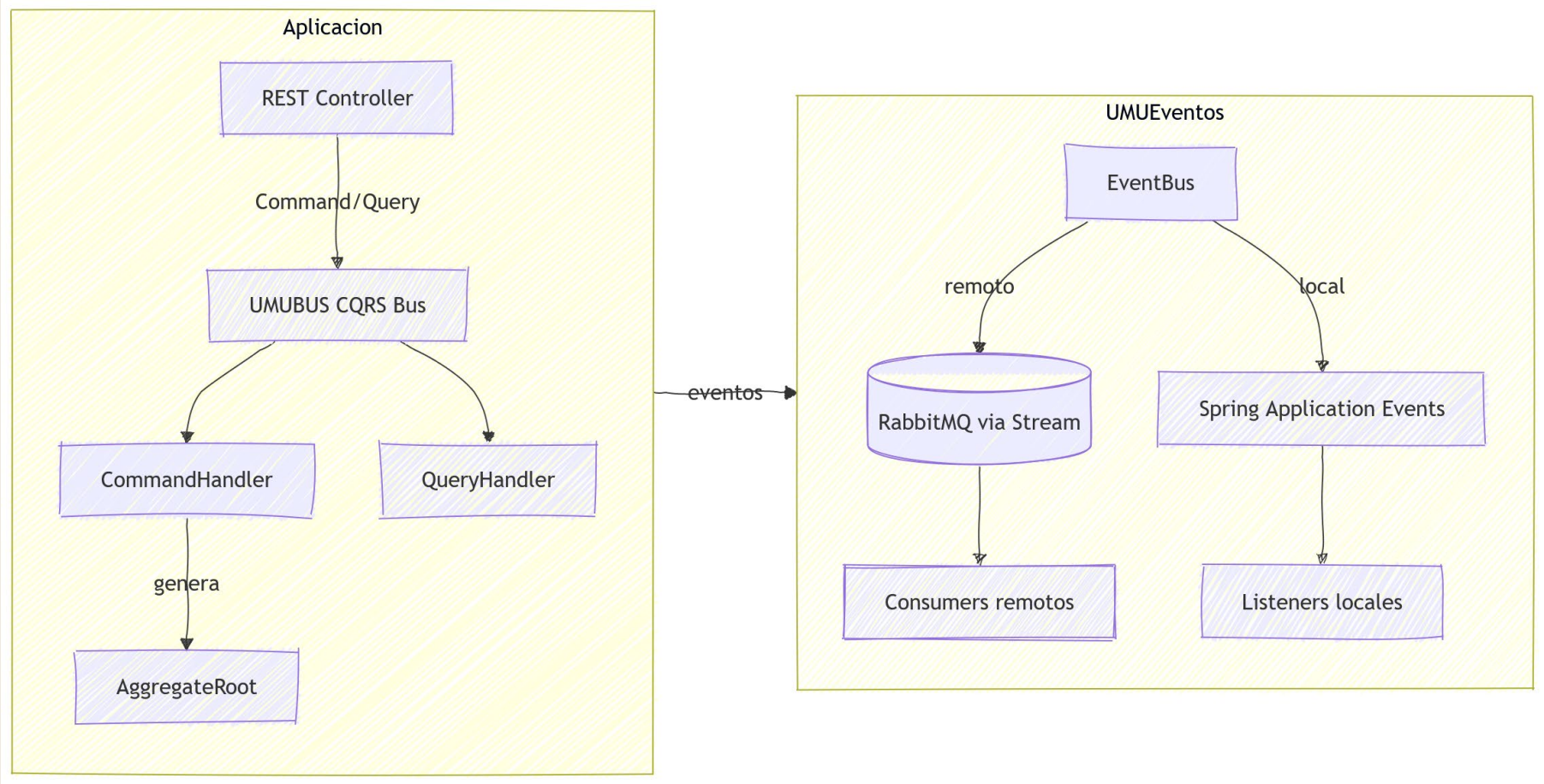
UNIVERSIDAD DE MURCIA

· 1272 ·

Apéndice 1: UMUBUS - ¿Qué es? ¿Para qué la usamos?

- Librería interna para estandarizar e implementar:
  - CQRS en local: **commands** (sync/async) y **queries** (sync)
  - **Eventos**: local y remoto (Rabbit), sync/async
  - Manejo de errores de mensajería: duplicados, reintentos, DLQ/fallback
- Estándar común en los proyectos backend de la Universidad
- Nos permite:
  - Unificar cómo definimos casos de uso (Query/Command y Handlers)
  - Desacoplar controladores de la lógica de aplicación (QueryBus/CommandBus)
  - Utilizar eventos locales/remotos y solucionar problemas derivados de su uso.

Apéndice 1: UMUBUS - CQRS + EventBus en UMUBUS



Apéndice 1: UMUBUS – Incluir la librería

- **Dependencia en pom.xml**

```
<repositories>
        <repository>
                <id>fundewebjs.archiva.atica.umu.es</id>
                <name>ATICA - UMU Repository - FundeWebJS</name>
                <url>https://archiva.um.es/archiva/repository/FundeWebJS/</url>
                <releases>
                        <enabled>true</enabled>
                </releases>
                <snapshots>
                        <enabled>true</enabled>
                </snapshots>
        </repository>
</repositories>
```

Apéndice 1: UMUBUS – Incluir la librería (Eventos locales)

- **Dependencia base en pom.xml**

```
<!-- UMUBUS local (CQRS + eventos locales) -->
<dependency>
        <groupId>es.um.atica.fundewebjs</groupId>
        <artifactId>fundewebjs-umubus</artifactId>
        <version>1.0.15-SNAPSHOT</version>
</dependency>
```

- **Recordar añadir en la clase principal de nuestra app el escaneo de paquetes**

```
@SpringBootApplication(scanBasePackages = { "es.um.atica.[CONTEX]", "es.um.atica.fundewebjs"})
```

Apéndice 1: UMUBUS - Sync vs Async (semántica)

- **Async Command (Command.java)**
  - “Solicito hacer X”
  - El bus ejecuta el handler y no devuelve respuesta de negocio
  - La confirmación (si aplica) llega como evento
- **Sync Command (SyncCommand.java)**
  - Solicito hacer X” y quiero un resultado inmediato
  - El handler devuelve un T (resultado)

Apéndice 1: UMUBUS – Clase Command (Usuario)

```
public class EliminarUsuarioCommand extends Command {
   private String id;
         //Definimos constructor privado
   public static EliminarUsuarioCommand of(String id) {
      return new EliminarUsuarioCommand(id);
   }
}
```

Apéndice 1: UMUBUS – Clase CommandHandler 1 (Usuario)

```
@Component
public class EliminarUsuarioCommandHandler implements
CommandHandler<CrearUsuarioCommand>{
   private UsuarioWriteRepository usuariosWriteRepository;
   private EventBus eventBus;          //Esto es nuevo
         //Definimos constructor
   @Override
public void handle(EliminarUsuarioCommand command) {
......
}
```

Apéndice 1: UMUBUS – Clase CommandHandler 2 (Usuario)

```
public void handle(EliminarUsuarioCommand command) {
   // Logica de negocio, llamada al dominio y...normalmente publicar evento final
          u.deleteUsuario();
          eventBus.publish(u);
}
```

- En el adaptador REST, ya no podemos llamar directamente al handler, al ser asíncrono tenemos que usar polling (BUS)

```
commandBus.handle(new CrearReservaCommand(...));
```

Apéndice 1: UMUBUS – Eventos locales VS remotos

- **Evento local**
  - Dentro del mismo proceso
  - Útil para desacoplar módulos internos
- **Evento remoto (Rabbit)**
  - Cruza proceso/red
  - Requiere resiliencia: duplicados, retries, DLQ/fallback

- UMUBUS proporciona soporte eventos local/remoto, sync/async y problemas típicos

Apéndice 1: UMUBUS – Implementación de eventos

- Clase que extiende de **Event**
- **Event.java** contiene:
  - **type** (autogenerado)
  - **version** (control de versiones, por defecto “1”)
  - **isLocal** (indica si el evento es local o remoto, por defecto “false”)
  - **metaData** (se genera automáticamente. Mapa con atributos de instancia)
- Antes de publicarlo, UMUBUS llama a **event.setMetaData()** para generar **metaData** con los atributos/valor del evento.
- Luego, el commandHanderl publicará todos los eventos creados por el Agregado raíz con eventBus.publish(aggregate).

Apéndice 1: UMUBUS – Implementación de eventos (Continuación)

- El **type** se genera así. Ejemplo:
  - Clase es.um...ReservaCreadaEvent
  - type = events.1.es.um....ReservaCreadaEvent
- El **type** es la “routing key semántica” del evento.
- En remoto, el type suele usarse para enrutar consumidores.
- Crear una clase evento:
  1. Crear clase XxxEvent extends Event
  2. Añadir solo los campos que quieres transportar
  3. (Opcional) definir versión en super(version)
  4. (Opcional) marcar como local con setLocal(true)

Apéndice 1: UMUBUS – Clase event (EliminarUsuarioEvent)

```
public class EliminarUsuarioEvent extends Event {
   private String usuarioId;

   private EliminarUsuarioEvent() {
            super();
   }

   private EliminarUsuarioEvent(String id) {
            super();
      this.usuarioId = id;
   }
}
```

Apéndice 1: UMUBUS – Publicación de eventos (AggregateRoot)

- Para poder realizar la publicación de eventos, nuestro Agregado raíz deberá extender de AggregateRoot(...)
- El AggregateRoot mantiene una colección de eventos y se añaden con addEvent(...)
- La operación a realizar del agregado raíz deberá añadir el evento correspondiente.
- Luego, el commandHanderl publicará todos los eventos creados por el Agregado raíz con eventBus.publish(aggregate).

Apéndice 1: UMUBUS – Clase AggregateRoot (Usuario)

```
public class Usuario extends AggregateRoot {
   private String id;
         ....
   public void createUsuario() {
      this.addEvent(CrearUsuarioEvent.of(this));
   }
   public void deleteUsuario() {
      this.addEvent(EliminarUsuarioEvent.of(this));
   }
}
```

Apéndice 1: UMUBUS – Consumir eventos locales (Spring)

```
@Component
public class EliminarUsuarioConsumer {

        @EventListener
        public void accept(EliminarUsuarioEvent event) {
         // Logica de negocio, crea eventos de respuesta OK/KO
   }
}
```

Apéndice 1: UMUBUS – Incluir la librería (Eventos remotos)

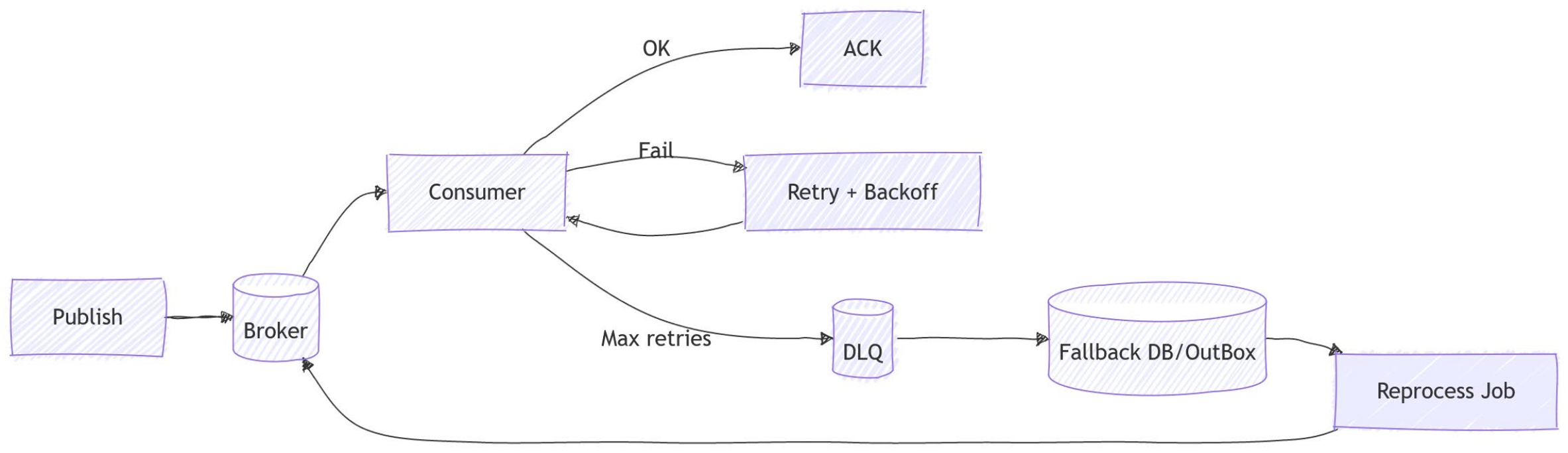
- **Dependencia rabbit (incluye el base) en pom.xml**

```
<!-- UMUBUS local (CQRS + eventos locales) -->
<dependency>
        <groupId>es.um.atica.fundewebjs</groupId>
        <artifactId>fundewebjs-umubus-rabbit</artifactId>
        <version>1.0.15-SNAPSHOT</version>
</dependency>
```

Apéndice 1: UMUBUS – Consumir eventos remotos (Rabbit)

- No vamos a entrar en detalles, pero....
  - Implementa un EventBus remoto (Rabbit) y sustituye el local Añade:
    - Productor (Supplier) para publicar en Rabbit mediante Spring Cloud Stream
    - Consumidor de ACK + job de reproceso (fallback)
    - Persistencia de mensajes fallidos en BBDD (fallback store)
- La diferencia es que, si el evento no es local, se encola para enviar a Rabbit
- Si el evento es local, se publica por ApplicationEventPublisher
- **Consumer remoto** (Spring Cloud Function) y configuración de spring.cloud.function.definition + bindings.
- UMUBUS contempla:
  - Retries
  - Dead-letter
  - Fallback/circuit breaker ante fallos de red
  - Persistencia de mensajes fallidos para reintento posterior

Apéndice 1: UMUBUS – Diagrama completo UMUBus



UNIVERSIDAD DE MURCIA

· 1272 ·